

Artificial Intelligence and Music: Implementing an Interactive Computer Performer

Author(s): Bridget Baird, Donald Blevins and Noel Zahler

Source: *Computer Music Journal*, Summer, 1993, Vol. 17, No. 2 (Summer, 1993), pp. 73-79

Published by: The MIT Press

Stable URL: <https://www.jstor.org/stable/3680871>

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



The MIT Press is collaborating with JSTOR to digitize, preserve and extend access to *Computer Music Journal*

JSTOR

Bridget Baird, Donald Blevins, and Noel Zahler

The Center for Arts and Technology
Connecticut College
New London, Connecticut 06320 USA
nbzah@conncoll.bitnet

Artificial Intelligence and Music: Implementing an Interactive Computer Performer

When live performers play music together, they interact with one another as they play, making adjustments in dynamics, in tempo, etc. The artificially intelligent computer performer is a software program that enables a computer to emulate a live performer. The computer listens to a live performer and matches the incoming information to the score; it then makes a decision about where the live performer is in the score and what the appropriate response should be. The computer plays its own part and adjusts its tempo and dynamics to keep pace with the live performer. The primary goal of this research has been to study the human cognitive abilities that interpret and process music. The heart of the program is the pattern-matching algorithm that the computer uses to analyze input; real-time processing considerations and anticipation of the kinds of mistakes that human performers are most likely to make have contributed to its development.

Background

The use of computers in live musical performance has a long history. Computers have been used to generate and perform compositions and have often been accompanied in these performances by humans. In many of the previous human-computer musical collaborations, the computer was used during the actual performance solely as a versatile tape machine—playing its part (which could include multiple instruments and lines) with admirable virtuosity but in blissful ignorance of the circumstances of the performance and its fellow musicians. The goal of our work has been to extend this capability, to make the computer not only a solo virtuoso but also an accom-

plished ensemble player, thus enabling us to learn more about human, musical, cognitive processes. In essence we would like to give a computer the talents necessary to become a fully participating member of a jazz ensemble, chamber music group, or rock band.

These musical talents include not only the ability to produce technically correct or even brilliant performances, but also the ability to respond to fellow performers and to adapt to the external variables of a performance. A good ensemble player must constantly monitor the performance of fellow musicians, adjusting tempo and dynamics to stay in step with the group. He/she must be prepared for mistakes of various kinds from the other performers, must not be confused and misled by those mistakes, and sometimes must make intentional alterations in the performance to compensate for the mistakes of others. The computer performer should simulate these abilities.

In its simplest outline, our computer performer will hold a full musical score in its memory. It will then “listen” to a single human performer, compare the incoming information with this score, make decisions about the progress of the performance, and if necessary adjust its own performance to achieve a correct, as stipulated by the score, musical result.

Researchers in psychoacoustics are actively engaged in studying how humans recognize pitch from ambient, analog sources. Although these problems are interesting and challenging, they are peripheral to our considerations. We therefore have chosen to implement the “listening” capabilities of our computer performer through the MIDI specification. The MIDI input provides our computer performer with pitch, duration, and dynamics data for each note produced by the human performer. The computer must take this sequence of data and attempt to match it with a sequence of notes in the score. This task, even for a “perfect” performance by a human, is more dif-

Computer Music Journal, 17:2, pp. 73–79, Summer 1993
© 1993 Massachusetts Institute of Technology.

ficult than it might seem at first glance. If a quarter note is supposed to last 0.5 sec, then the computer should not view as a mistake a note which is heard to last 0.51 sec. Similarly, if two notes for a keyboard instrument are written in the score as consecutive, the computer should not interpret as a rest the pause while the human performer moves a finger from one key to another. In effect, the computer must be less precise than its capabilities. When the computer can correctly interpret the incoming data stream as a sequence of whole notes, half notes, rests, etc., the next task is to determine which portion of the score this sequence of notes is intended to represent, even if the performer has made some mistakes in the tempo and/or pitch of some of the notes, or perhaps has even become disoriented in the score. The computer needs to decide where in the score the human is playing and at what tempo and dynamic he/she is playing. Then, on the basis of this decision, it must adjust its own performance to conform. Again, in this task the computer must be in effect hobbled and made to make changes on a human time scale rather than on a computer time scale in order to sound musical and not harsh and mechanical.

Overview

Several researchers in artificial intelligence and music have built computer systems that allow the computer to accompany a live performer. At Carnegie Mellon University, Dannenberg (1984) and Bloch and Dannenberg (1985) have developed a computer accompaniment system that follows a live performer; it matches input to the score by considering pitches and building a cost estimate of matches to the score. Dannenberg and Mukaino (1988) introduced some parallelism into this tracking algorithm by considering competing hypotheses in the score. This system has been implemented on Amiga and IBM micro-computers.

At MIT, Vercoe (1984) and Vercoe and Puckette (1985) developed a similar system that uses pitch and attack time to make matches and also incorporates musical memory into the computer by having it keep track of previous performances. This system

runs on a Digital Equipment Corporation DEC VAX computer and has recently been ported to an Apple Macintosh II. The tracking algorithms for these systems are quite different, but both Dannenberg's and Vercoe's algorithms use pitch and time to make decisions.

The Dannenberg matcher uses pitch as the single most important criterion for synchronizing the computer accompanist with a live performer. This matcher is derived from the dynamic programming algorithm for finding the longest common subsequence of two strings of pitches (Sankoff and Kruskal, 1983). One string represents the score, which has been preloaded in the computer's memory, and the other is the input from the live performer. The algorithm eliminates arbitrary characters (pitches) from each string until the remaining characters (subsequences) match exactly (Dannenberg 1984). Dannenberg modifies this algorithm by shortening the strings. Considering the entire live performance and score strings is unnecessary and computationally inefficient for the purposes of real-time accompaniment. Instead, a moving window is used to reduce the length, thus only a portion of the data is considered at any one time. What needs to be known is the element of the score, if any, to which each element of the performance corresponds. Assuming that the performance is fairly accurate, the longest possible common subsequence within the window is presumed to be the correct match. This "best guess" as to the performer's place in the score is designated by a heuristic.

Vercoe's algorithm for matching (Vercoe and Puckette 1985) computes how "closely a theory about the current tempo and score position fits the incoming data." A cost is assigned that measures the deviation of the actual performance from the score. This cost charges for combinatorial errors—notes in the score that are missing from the performance, extra notes, and wrong notes. A cost is also added for metrical errors. The metrical cost of the theory is measured as the deviation of the performer from a weighted least-squares fit of a line through the points (s_i, t_i) , where s_i is a point in time associated with the score and t_i is the time the performer actually executed the event. If points are dropped out of this fit,

there is an additional cost. "The cost of the theory is the sum of all the above charges. The problem of tracking the performance is that of finding the least costly theory." Information from the least-squares fit is used to "predict what the performer will do next or to schedule the performance of an accompaniment."

In 1988, the authors of this article began work on a software computer performer. The work is being done on a Macintosh II and is written in C. Musical input and output occur through a synthesizer connected to a MIDI interface. This means that the computer receives digital information about pitch, amplitude, duration, and attack time of notes being played. The computer has, in its memory, the score of the composition. The score is entered via standard MIDI formats using readily available commercial software. The musical style of the score is not a limiting factor (the authors have tested the computer performer on several different styles), but the computer does require a single monophonic line of input from the human performer. The computer performer plays its part or parts through a synthesizer, also using MIDI format. The program has Macintosh-style menus, with the ability to record performances, store them, and recall them. This makes evaluating the performance of the program much easier. The program performs a preperformance score consultation to determine the meter, tempo, and key of the piece and to flag tonic cadences. If, during a performance, the computer performer concludes that the live performer is lost, the computer may use these cadences to cue the live performer as to position in the score. The program also monitors the volume level of the performance and adjusts its own dynamic level accordingly.

The original implementation of the computer performer could use either Dannenberg or Vercoe's tracking algorithms, but modified them with heuristics about tempo adjustment and dynamics (volume). This original implementation was presented at the Connecticut College Arts and Technology Symposium (Baird, Blevins, and Zahler 1989a) and at the European Workshop on Artificial Intelligence and Music (Baird, Blevins, and Zahler 1990).

Human musical performers track musical compo-

sitions by relying on patterns of notes, the duration of notes, absence of notes (rests), and likely mistakes of performers. The authors believed that the algorithms of Dannenberg and Vercoe did not reflect these musical cognitive processes and wished to explore ways that would more accurately do this. They began to develop a new tracking algorithm and demonstrated the computer performer using this algorithm at the 1989 International Computer Music Conference (Baird, Blevins, and Zahler 1989b). These results and further development of the tracking algorithm and the kinds of tempo adjustments that the program subsequently makes are discussed more fully below.

Tracking Algorithm

Tracking a live performance is a pattern-matching problem. In making this pattern-match, the authors were guided by the actions of musicians. Although both Dannenberg's and Vercoe's algorithms worked well in performance, they both used pitch and attack time as the indicators of place. People recognize sequences of notes, and these sequences take into account pitch, length of the note, and rests. For example, most people who have listened to Western music are able, in four notes, to recognize *Twinkle, Twinkle Little Star*



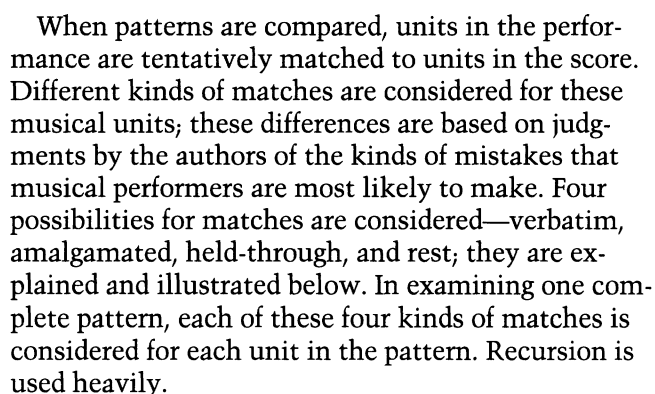
and are likewise able to place the central theme of the first movement of Beethoven's Fifth Symphony (or at least know that it is familiar) from its opening musical motive



In recognizing musical motives, players are guided by style and context.

The authors' tracking algorithm begins to simulate

Since this tracking algorithm takes into account durations of notes and rests, and since musicians consider this information continuously, the tracking algorithm is called whenever a new note or rest begins and also at regular intervals during notes or rests. The length of this interval, as well as the period of elapsed time before the program decides that the performer is playing a rest (as opposed to merely pausing between notes), is determined in the preperformance score consultation and depends on the meter and tempo of the piece. When the tracking algorithm is invoked, a single performance pattern, which consists of the most recently heard notes or rests, is established. This performance pattern is compared with several possible sequences of notes (patterns) in the score. Ideally the program would consider all possible score patterns, but as this is currently not feasible because of time constraints, a window into the score limits the possibilities. This window is centered on the previous best match



Score



Live Performer

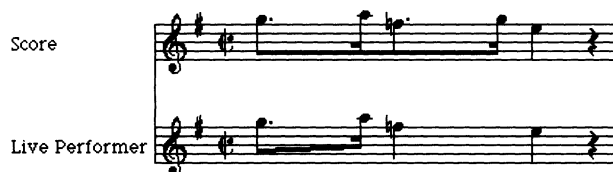


The figure shows two staves of music. The top staff is labeled 'Score' and the bottom staff is labeled 'Live Performer'. Both staves contain the same musical notation, which is a short melody in G major (one sharp) and 4/4 time. The melody consists of the notes G4, A4, B4, A4, G4, F#4, E4, and D4, followed by a quarter rest. The notation is identical for both the Score and Live Performer parts.

The image shows two staves of musical notation. The top staff is labeled 'Score' and the bottom staff is labeled 'Live Performer'. Both staves are in G major (one sharp) and 4/4 time. The notation is identical for both: a quarter note G4, a quarter note A4, a quarter note B4, a quarter note G4, a quarter note F#4, a quarter note E4, and a whole note D4.

Held-through—one note from the performer is matched to two notes in the score. The performer has elected, for reasons of tempo or difficulty of performance, not to play a note in the score and has de-

cided, instead, to hold the previous note through the following one. Two units from the score are treated as a single entity, with pitch taken from the first, and duration the combination of the two, as in



Rest—the performer releases a note early. One puzzle facing the authors was the decision on what length of time constitutes a rest, rather than a pause between notes. This parameter varies from piece to piece and is set in the preperformance score consultation. Regardless of how it is set, there may be times in a piece when an incidental pause by the performer is longer than an intentional rest. The last pattern match allows for this possibility. A note and the subsequent pause are joined into one unit, and this unit is compared to one note in the score. The pitch of the performer's note is matched to the pitch of the note in the score, and the duration of the performer's unit is the sum of the duration of the note played and the subsequent pause.



The computer performer must make pattern matches in real time. If the computer takes too long to make a decision, then its response is outdated. As the program compares the performance pattern to a possible score pattern, it builds up a cost for the match (lower costs are preferable). An upper limit on costs for acceptable matches is established, and when a pattern match exceeds this cost, it is discarded as a possibility. The program has a default built in for cases where no acceptable match is found. The default is basically "keep playing" while gradually reverting to the original tempo of the piece.

When a performance pattern is to be matched, the algorithm first makes a guess, based on previous

score position, for the most promising score pattern. All subsequent matches to score patterns use the information from the previous best guess; as soon as a new cost exceeds the previous best guess, that path is abandoned. Both the size of the window (hence the number of score patterns to be considered) and the length of the performance pattern affect the speed of the algorithm. The authors experimented with both of these parameters and augmented them until degradation in response was noticeable. At present the size of the window is set at five notes and the size of the performance pattern is at a maximum of seven notes. The size of the pattern is also restricted by stylistic considerations: for example, in a piece with a very slow tempo, a pattern size of seven is far too large. This size is limited by a time interval that is set at the beginning of the piece in a preperformance score consultation.

Tempo Adjustment

During the performance the computer performer is constantly adjusting its tempo to match that of the performer. If a good match is established during the tracking algorithm, but the computer performer is not playing at the location identified by this match (the computer performer and the live performer are in different places), then the computer performer must synchronize with the live performer. In order to do this there must be a good estimate of the performer's current tempo; the computer performer needs to know not only where the performer is in the score (given by the tracking algorithm) but also how fast the performer is playing. A history of recent matching positions and of the times of the matches, is kept by the computer performer. As a new match comes in, a weighted least squares fit (with greater weight on more recent information) is computed in order to arrive at a best estimate of the live performer's tempo.

Once the computer performer has estimated the live performer's position and tempo, it must decide on an appropriate response. The computer performer may decide to jump to the performer's position or may elect to speed up or slow down in order to mesh with the performer. An immediate jump to the cor-

rect position in the score sounds very unmusical to the ear and is used only in extreme situations. Instead, an interval over which to catch the performer is used to adjust the tempo. Adjustments need to be made fairly quickly, but ones that are too fast are jarring to the ear. The authors experimented with different intervals and chose a size of roughly 2 sec (with some variations due to the tempo of the piece). The degree of speeding up or slowing down is also dependent on the present tempo; abrupt changes are excluded.

As the computer performer adjusts its tempo other factors are considered. In the pre-performance score consultation, absolute minimum and maximum tempi are established. These are dependent on the tempo and meter of the piece. In a very slow piece, for example, an extremely fast tempo is unacceptable. There is more leeway in slowing down than in speeding up; an extreme slowing of the tempo amounts to silence or long held notes on the part of the computer performer.

If there is no acceptable estimate of the performer's position in the score, then the computer performer keeps playing. What should the tempo adjustment be in such situations? To maintain the last tempo of the computer performer is not a good choice since that tempo may have reflected a slowing down or speeding up. To play at the last tempo of the live performer is a possibility, but as no new good matches are made, this information becomes outdated. The program uses the original tempo of the piece as a guide and gradually reverts to it until new acceptable matches are found.

Ongoing and Future Work

The computer performer is being implemented to respond to input from several live performers. This implementation uses parallel processing via transputers, which are microprocessors that operate independently and concurrently and are installed in the Macintosh. Each transputer receives information from the Macintosh about a single live performer, performs the pattern match outlined above, and then sends back the best estimate of the player's position in the score and an evaluation of the merit of that es-

timation. The Macintosh then reconciles all of the information from the different transputers to decide on a single best position and on a single best tempo response. This reconciliation of possibly conflicting information from the transputers involves a weighting that takes into account when the information was received (instruments that have been most recently heard have greater weight), how good the pattern match is, and the importance or prominence of different musical lines. This process of reconciliation is a complicated one, and we are exploring it further.

One fascinating area of research is the description and use of musical motives. This would be important for both the monophonic input and the input from multiple live performers. The authors chose to match musical patterns because they believe that this is a more accurate reflection of the way musicians play music. The musical patterns that the algorithm uses are determined by time and by size. Ideally they should also be determined by musical considerations. Musical patterns should not bridge breaks in the score and should form coherent musical phrases. Considerations of this kind will, of course, depend on the style of the music. There has been much research in musical cognition on this subject, and some of these results should be incorporated into the tracking algorithm.

For beginning musicians, the kinds of mistakes that are made are quite different from those of a more advanced musician. The computer performer could be modified to play with different types of players and to present this choice during the preperformance consultation. Beginning musicians make different kinds of pitch mistakes and tend to go back and repeat a misplayed passage. The tracking algorithm could be developed to reflect this kind of situation. Tempo adjustments would also be different; the computer performer could adhere more strictly to the original tempo and thus assume more of the role of a teacher.

Conclusion

At this point, the computer performer responds extremely well when performing with a live performer who plays in a reasonably accurate manner. The

computer performer is quite tolerant of mistakes if the live performer's mistakes include wrong notes and if the live performer moves ahead in the piece, rather than replaying passages. If the live performer changes tempo, the computer performer will follow and accept the new tempo. In fact, the computer performer is very good at following tempo changes, as long as they are not abrupt, and this ability gives a real feeling of interactive ensemble playing. The computer performer is definitely a follower, not a leader, and will not improvise. Except for changes in tempo, it plays its part the same way each time a piece is performed. It does not have to "learn" a piece, but, conversely, it has no memory of a previous performance. Thus with good musicians, the computer performer is a very good accompanist.

References

- Baird, B., D. Blevins, and N. Zahler. 1989a. "The Artificially Intelligent Computer Performer." In *Proceedings of The Second Symposium on Arts and Technology*. New London, Connecticut: Connecticut College Press, pp. 16–23.
- Baird, B., D. Blevins, and N. Zahler. 1989b. "The Artificially Intelligent Computer Performer on the Macintosh II and a Pattern Matching Algorithm for Real-time Interactive Performance." In *Proceedings of the 1989 International Computer Music Conference*. San Francisco: International Computer Music Association, pp. 13–16.
- Baird, B., D. Blevins, and N. Zahler. 1990. "The Artificially Intelligent Computer Performer: The Second Generation." *Interface: The International Journal of Technology and the Arts* 19.
- Bloch, J. J., and R. B. Dannenberg. 1985. "Real-Time Computer Accompaniment of Keyboard Performances." In *Proceedings of the 1985 International Computer Music Conference*. San Francisco: International Computer Music Association, pp. 279–289.
- Dannenberg, R. B. 1984. "An On-line Algorithm for Real-Time Accompaniment." In *Proceedings of the 1984 International Computer Music Conference*. San Francisco: International Computer Music Association, pp. 193–198.
- Dannenberg, R., and H. Mukaino. 1988. "New Techniques for Enhanced Quality of Computer Accompaniment." In *Proceedings of the 1988 International Computer Music Conference*. San Francisco: International Computer Music Association, pp. 243–249.
- Sankoff, D., and J. B. Kruskal, eds. *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison*. Reading, Massachusetts: Addison-Wesley, 1983.
- Vercoe, B. 1984. "The Synthetic Performer in the Context of Live Performance." In *Proceedings of the 1984 International Music Conference*. San Francisco: International Computer Music Association, pp. 199–200.
- Vercoe, B., and M. Puckette. 1985. "Synthetic Rehearsal: Training the Synthetic Performer." In *Proceedings of the 1985 International Computer Music Conference*. San Francisco: International Computer Music Association, pp. 275–278.